

nodeIO_master — README del programador

Pasarela **LoRa** ↔ **Modbus RTU** para **Heltec WiFi LoRa 32 V3** (ESP32-S3 + SX1262).

Sondea hasta 8 nodos **nodeIO** por LoRa y los expone como un **esclavo Modbus RTU** en RS-485. El descubrimiento y la adopción de nodos se hacen desde su portal cautivo.

Antes fue un clon de `../nodeIO`; hoy solo comparte `src/io.{h,cpp}` y `src/images.h` byte a byte.

1. Toolchain

```
pio run
pio run -t upload
pio device monitor -b 115200      # solo log; Modbus va por RS-485
```

`platformio.ini` → `lib_deps`: `ropg/Heltec_ESP32_LoRa_v3`, `bakercp/CRC32`, `emelianov/modbus-esp8266`.

2. Arquitectura

Módulo	Responsabilidad
<code>src/main.cpp</code>	Estados <code>MODE_NORMAL</code> / <code>MODE_PORTAL</code> , splash, OLED de estado, long-press de <code>BUTTON_1</code> . Único TU que incluye <code>heltec_unofficial.h</code>.
<code>src/master_config.{h,cpp}</code>	<code>MasterConfig</code> en NVS (namespace <code>masterio</code> , blob + magic): LoRa, Modbus, polling, IO local, AP y tabla de nodos <code>MasterNode nodes[8]</code> . Helpers <code>masterFindByMac/Addr</code> , <code>masterAddNode</code> , <code>masterRemoveNode</code> , <code>mbFormatToConfig</code> .
<code>src/lora_master.{h,cpp}</code>	Lado LoRa: <code>masterBegin()</code> , <code>masterPollLoop()</code> (round-robin <code>RD</code> , cola de escritura <code>WR/WP</code> , marcado online/offline en <code>snap[8]</code>), y los bloqueantes <code>masterDiscover()</code> / <code>masterAdopt()</code> / <code>masterRelease()</code> usados por el portal.
<code>src/modbus_gw.{h,cpp}</code>	<code>ModbusRTU</code> (esclavo) sobre <code>Serial1</code> . <code>modbusBegin()</code> da de alta el mapa (8×16 + bloque global). <code>modbusTask()</code> = <code>mb.task()</code> + publicación throttled de <code>snap[]</code> . Callbacks <code>onSetCoil</code> → <code>masterQueueRelays/Pulse</code> .
<code>src/portal_master.{h,cpp}</code>	Portal cautivo (DNS :53 *, <code>WebServer</code> :80). Rutas <code>/save</code> / <code>/scan</code> / <code>/adopt</code> / <code>/remove</code> / <code>/toggle</code> . La radio sigue viva en modo portal para descubrir/adoptar.

Módulo	Responsabilidad
<code>src/io.{h,cpp}</code>	Copia de <code>nodeIO</code> (pines/ISR/relés). Aquí solo se usan los botones y, si <code>localIoEnabled</code> , las lecturas AI/DI.
<code>lora_master.cpp</code> y <code>modbus_gw.cpp</code> incluyen <code><RadioLib.h></code> / <code><ModbusRTU.h></code> y <code>extern SX1262 radio;</code> (la instancia global vive en <code>heltec_unofficial.h</code>, incluido solo por <code>main.cpp</code>). Dependencias entre módulos: <code>modbus_gw</code> → <code>lora_master</code> (lee <code>snap[]</code>, encola escrituras). <code>portal_master</code> → <code>lora_master</code> + <code>master_config</code>. Sin ciclos.	

3. Flujo

```

setup(): heltec_setup(); masterConfigLoad(); ioInit(0,0); splash();
        nodeCount == 0 ? { masterBegin(); enterPortal(); }
                        : { masterBegin(); modbusBegin(); MODE_NORMAL }

loop(): heltec_loop();
        BUTTON_1 > 3 s -> enterPortal()  (para polling+Modbus; radio sigue
viva)
        MODE_PORTAL: portalLoop(); drawPortalScreen(); return;
        MODE_NORMAL: masterPollLoop(); modbusTask(); drawStatusScreen();

```

`masterPollLoop()` es una máquina de estados no bloqueante: cada `pollMs` avanza al siguiente slot habilitado y envía `RD` (o el `WR/WP` encolado); espera `ackTimeoutMs`; `offlineAfter` fallos → `snap[i].online = false`.

4. Protocolo LoRa

Misma trama que el nodo (ver `../nodeIO/PROTOCOL.md`), incluidos los añadidos de aprovisionamiento:

- `DISC` (→255) → el gateway recoge `IAM,<mac>,<fw>` de los nodos sin adoptar.
- `ADOPT,<mac>,<addr>,<freq>,<sf>,<bw>,<cr>,<sync>,<pwr>` (→255) → el nodo guarda dirección + canal y responde `ACK,<mac>`.
- `RELEASE,<mac>` → el nodo vuelve a "sin adoptar".
- Operación: `RD` → `ST,...; WR,<r1...r4>; WP,<idx>,<ms>`.

El gateway usa `mcfg.masterLoraAddr` (def. 200) como `src` y `mcfg.lora*` como canal autoritativo, que empuja en cada `ADOPT`.

5. Mapa Modbus

Definido en `modbus_gw.h`. Slave id `mcfg.mbSlaveId`. Bloque por nodo `i = 0..7`, base `i*16`: Input Reg (AI/DI/relés/link/RSSI/edad/addr), Discrete Inputs (DI + link), Coils (consigna de relé + disparo de pulso). Bloque global en Ireg 900; bloque IO local (si `localIoEnabled`) en Ireg 904+ y Coil 900+.

6. Cómo extender

Nuevo campo de config: añádelo a `MasterConfig`, default en `masterConfigFactory()`, sube `CFG_MAGIC` en `master_config.cpp`, e input en `portal_master.cpp::buildPage()` + parseo en `handleSave()`.

Nuevo registro Modbus: amplía `NODE_BLK` o el bloque global en `modbus_gw`, dando de alta en `modbusBegin()` y publicando en `publish()`.

Otro comando LoRa saliente: añade un `sendFrame(addr, "XX,...")` y, si espera respuesta, un caso en `dispatchReply()`.

Multi-master: cada nodo solo obedece `ADOPT/RELEASE` con su MAC y, ya adoptado, filtra por `masterAddr`. Para mover un nodo, `masterRelease()` en un gateway y `masterAdopt()` en el otro.

7. Sincronización con nodeIO

Solo `src/io.{h,cpp}` y `src/images.h` deben mantenerse idénticos entre ambos proyectos. `platformio.ini` diverge (este añade `modbus-esp8266`). Todo lo demás es específico del gateway.

8. Notas de hardware

- UART1 Modbus por defecto en GPIO2/3 + DE GPIO4 = regleta AI del carrier; válido con `localIoEnabled = false`. Configurable en el portal.
- `mb.begin(&Serial1, dePin)` gestiona el DE/RE del RS-485; `dePin = -1` → UART TTL sin control de dirección.
- WiFi (2.4 GHz) y LoRa (915 MHz) son radios independientes: el portal mantiene la radio activa para descubrir/adoptar sin interferir.